

Software Architecture Documentation

1. Overview

This system enables AI-driven software testing by automatically generating test code using Large Language Models (LLMs). It is designed to simplify and accelerate the creation of test cases for existing software, primarily Python codebases, with a focus on UI, logic, and persistence layer testing (though currently demonstrated mainly with unit tests for Python functions).

The solution is built for on-premise operation, leveraging local LLM runtimes like Ollama to ensure data security and control. Users can interact with the system through two main interfaces:

1. **Web Interface:** A modern React-based chat interface that allows users to upload python files, select models and modules, and receive generated test code with real-time feedback.
2. **Command Line Interface:** A CLI tool for batch processing and automated workflows, supporting advanced features.

The system features a sophisticated **modular architecture** with 10+ specialized modules that can be combined to enhance the AI pipeline. These modules handle tasks like code complexity analysis, context size validation, test execution, code cleaning, and integration with external services.

By utilizing LLMs managed by Ollama, the system supports incremental test code generation and can adapt to code changes. The architecture includes features like dependency resolution, timeout handling, file upload capabilities, and comprehensive metrics collection, making it suitable for both interactive development and automated testing pipelines.

2. Key Components

The system is composed of several key components that work together:

1. Frontend (Web Interface):

- o **Description:** The primary user interface for the AI-Driven Testing system, developed as a **React/TypeScript Single Page Application (SPA)** with Material-UI components. It provides a modern chat-based interface enabling users to upload Python files, select Large Language Models (LLMs), configure processing modules, and interact with the AI system through natural language. The frontend features real-time model status monitoring, file upload capabilities, module dependency management, and comprehensive response display including timing metrics and complexity analysis.
- o **Location:** `frontend/` directory.

- **Key Components:**

- `App.tsx`: Main application component orchestrating the entire interface, managing state for models, modules, chat history, and file uploads.
- `components/ChatInput.tsx`: Input component handling user messages and Python file uploads (.py files), with file selection and validation.
- `components/ChatHistory.tsx`: Display component for conversation history showing user messages, AI responses, attached files, and response metrics.
- `components/TopBar.tsx`: Navigation bar with model selection dropdown, shutdown controls, and module sidebar access.
- `components/ModuleSidebar.tsx`: Advanced module management interface with dependency resolution, allowing users to select processing modules that automatically handle prerequisites.
- `components/PrivacyNotice.tsx`: Information display for model licensing and privacy information.
- `api.ts`: API client handling HTTP communication with the backend, including model management, module discovery, and prompt processing.

- **Key Files & Directories:**

- `public/`: Contains static assets accessible by the browser.
 - `index.html`: The main HTML shell into which the React application is injected.
 - `manifest.json`: Web App Manifest enabling Progressive Web App (PWA) features.
 - `favicon.ico`, `robots.txt`: Standard web assets.
- `src/`: This is the core directory containing all the application's source code, primarily React components written in TypeScript (.ts, .tsx files).
- `package.json`: Defines project metadata, scripts (e.g., `npm start` for development, `npm run build` for production builds), and lists all Node.js dependencies (e.g., React, Material-UI, Emotion, TypeScript, react-markdown).
- `package-lock.json`: Ensures reproducible installations of Node.js dependencies by locking their versions.
- `tsconfig.json`: The TypeScript compiler configuration file, specifying how TypeScript code is checked and transpiled.
- `Dockerfile`: Contains instructions to build a production-ready Docker image for the frontend. This typically involves building the static assets (using `npm run build`) and then serving them with a lightweight web server like `serve`.
- `.gitignore`: Specifies files and directories (like `node_modules/`, `build/`) to be ignored by Git version control within the frontend subdirectory.

2. Backend API (FastAPI Application):

- **Description:** A Python-based API built with FastAPI, running in a Docker container. It serves as the central hub, receiving requests from the Frontend and CLI. It orchestrates LLM interactions via the `LLMManager` and applies sophisticated pre/post-processing to prompts and responses using the `ModuleManager`. The API features automatic module discovery, dependency resolution, and comprehensive error handling.

- **Location & Key Files:** `backend/api.py` (FastAPI app), `backend/schemas.py` (Pydantic models), `backend/Dockerfile`.
- **Key Endpoints:**
 - `GET /models`: Returns list of available LLMs with their running status, licensing information, and metadata.
 - `GET /modules`: Auto-discovers and returns all available processing modules with their capabilities, dependencies, and documentation.
 - `POST /prompt`: Main processing endpoint that accepts user prompts, source code, model selection, and module configuration. Returns generated test code with timing metrics and module outputs.
 - `POST /shutdown`: Gracefully shuts down specific LLM containers to manage resource usage.
- **Key Features:**
 - **CORS Support:** Configured for frontend communication across different origins.
 - **Automatic Module Discovery:** Dynamically loads modules from the `modules/` directory with validation.
 - **Dependency Resolution:** Automatically resolves and loads module dependencies.
 - **Error Handling:** Comprehensive error handling with detailed error messages.
 - **Async Processing:** Uses FastAPI's async capabilities for concurrent request handling.

3. LLM Orchestration Layer (`LLMManager`):

- **Description:** A core Python class located in `backend/llm_manager.py`. It is responsible for the entire lifecycle management of **Ollama Docker containers** (one per active model). This includes pulling the base `ollama/ollama` image, pulling specific LLM models (e.g., Mistral) inside these containers, dynamically allocating free network ports, ensuring API readiness, sending processed prompts to the correct Ollama instance, and handling the streamed responses.
- **Location & Key Files:** `backend/llm_manager.py`.

4. Ollama Service (LLM Engine):

- **Description:** The actual engine that runs the Large Language Models. The project uses **Ollama** (via the `ollama/ollama` Docker image) to serve various open-source LLMs locally. Each selected LLM runs in its own Ollama container, managed by the `LLMManager`.
- **Configuration:** The list of supported LLMs and their Ollama IDs is defined in `backend/allowed_models.json`. The actual model data is persisted in the `backend/ollama-models/` directory (mounted as a Docker volume).

5. Test Generation Logic (LLMs & `ModuleManager`):

- **Description:** The core test code generation is performed by the **selected LLM** based on the (potentially pre-processed) prompt. The `ModuleManager` (defined in `backend/module_manager.py`) provides a sophisticated plugin architecture with **10+ specialized modules** that can be combined to create powerful AI processing pipelines. Each module can operate before LLM processing (preprocessing) and/or after LLM processing (postprocessing), with automatic dependency resolution and configurable execution ordering.

- **Location & Key Files:** `backend/module_manager.py`, `backend/modules/` (directory containing all processing modules).
- **Key Features:**
 - **Dependency Resolution:** Modules can declare dependencies on other modules, which are automatically loaded and executed in the correct order.
 - **Execution Ordering:** Modules can specify processing order priorities for precise control over the pipeline.
 - **Dynamic Discovery:** Modules are automatically discovered and validated at runtime with proper error handling.
 - **Snake/Camel Case Conversion:** Automatic naming convention conversion between file names and class names.
 - **Complex Workflows:** Support for iterative processing, code validation, test execution, and performance benchmarking.

6. Command Line Interface (CLI):

- **Description:** A comprehensive command-line interface providing advanced automation capabilities for batch processing and scripted workflows. The CLI supports all features available in the web interface plus additional capabilities like iterative refinement, custom output paths, and advanced module ordering controls. It's designed for CI/CD integration and power users who prefer terminal-based interaction.
- **Location & Key Files:** `backend/main.py` (entry point), `backend/cli.py` (argument parsing), `backend/execution.py` (processing logic).
- **Key Features:**
 - **Model Selection:** Choose from available LLMs using numeric indices or model IDs.
 - **Module Support:** Specify multiple modules with automatic dependency resolution and configurable execution ordering.
 - **File Input:** Support for various input methods including files, stdin, or interactive prompts.
 - **Iterative Processing:** Multiple processing iterations for code refinement and improvement.
 - **Custom Output:** Configurable output file paths and naming conventions.
 - **Batch Processing:** Suitable for automated testing pipelines and CI/CD integration.

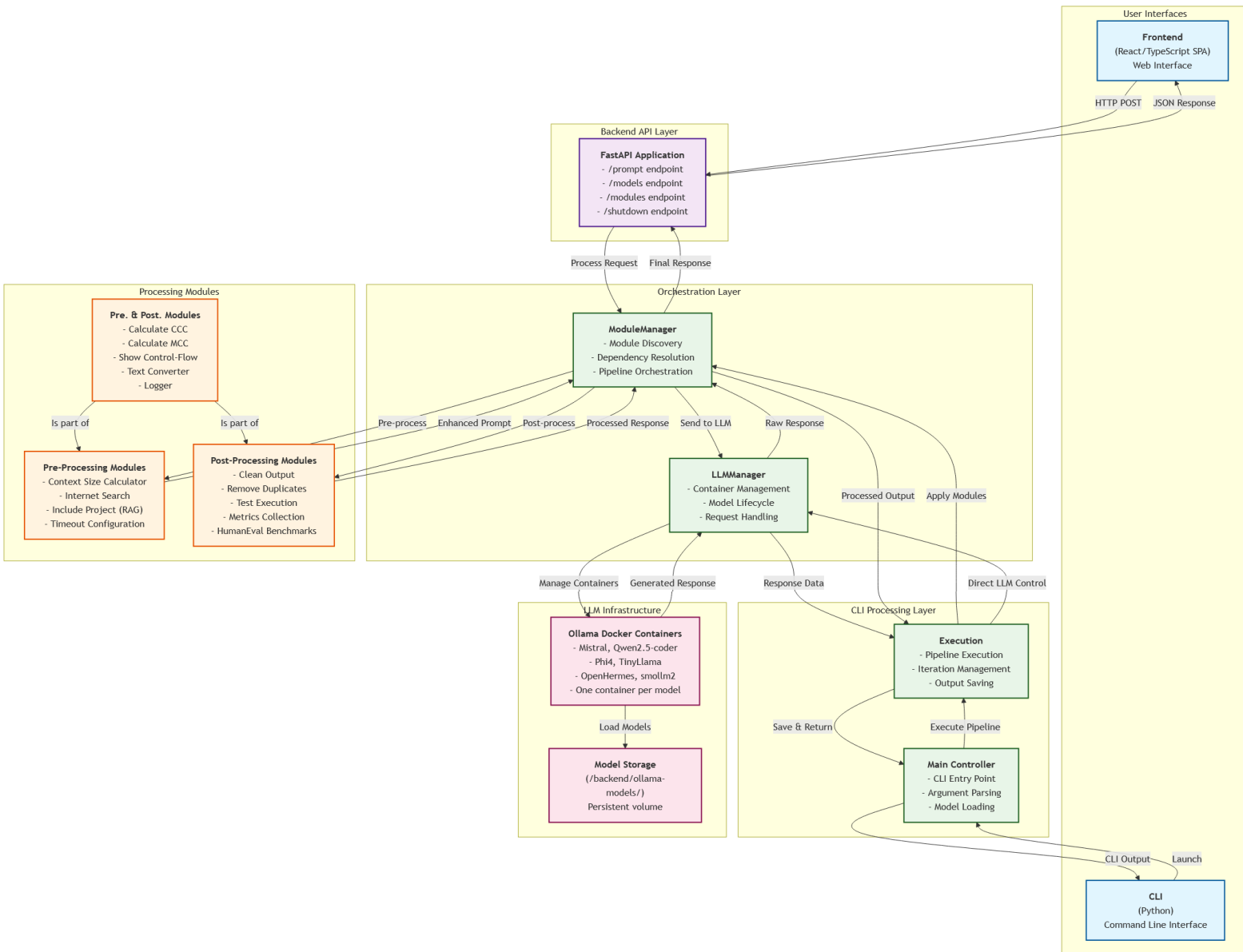
7. Output Handling & Storage:

- **Description:** LLM responses are primarily structured as Markdown. The backend, particularly through the flow orchestrated by `backend/execution.py`, saves the comprehensive `ResponseData` (which includes the Markdown output, model information, timing metrics, etc., as defined in `backend/schemas.py`) as structured JSON files. These are saved in both a timestamped archive (`outputs/archive/`) and a latest version (`outputs/latest/`). The Frontend is responsible for rendering the Markdown response to the user.
- **Location & Key Files:** `backend/execution.py`, `backend/schemas.py`.

8. Test Case Examples:

- **Description:** The system includes sample test cases and Python programs for demonstration and testing purposes. These are located in the `backend/python-test-cases/` directory and serve as examples of the types of tests the system can generate.

3. Architecture Diagram



(Simplified architecture diagram showing the clear separation between user interfaces, backend API, orchestration layer, processing modules, and LLM infrastructure.)

Interactive diagram viewer: <https://github.com/amosproj/amos2025ss04-ai-driven-testing/wiki/Architecture#3-architecture-diagram>

4. Technology Stack

Layer/Component	Technology / Tool	Purpose
Frontend	React, TypeScript, Material-UI, Emotion, React-Markdown	Modern chat-based UI with file upload, module management, and markdown rendering
Backend API	Python, FastAPI, Uvicorn, Pydantic	High-performance async web server with automatic API documentation
LLM Orchestration	Python, docker-py, requests, tqdm	Managing Ollama Docker containers, model lifecycle, progress tracking
LLM Engine	Ollama, Docker	Running various open-source Large Language Models locally
Data Management (Backend)	Pydantic (for schemas), JSON, Python datetime	Structuring and validating data, storing LLM responses with timestamps
Python Environment (Backend)	Conda with environment.yml, Python	Reproducible Python environment management
Test Generation (Core AI)	Various LLMs (Mistral, Qwen2.5-coder, Phi4, TinyLlama, Qwen3, OpenHermes, smollm2, StarCoder2 via Ollama - see <code>backend/allowed_models.json</code>)	Core AI-driven test code generation with multiple model support
Build/Orchestration (Overall)	Docker, Docker Compose	Containerization, multi-service application setup and management
Code Quality & Formatting	Black, Flake8, McCabe	Maintaining Python code standards and complexity analysis
Testing Frameworks	pytest, unittest, Jest (frontend), React Testing Library	Comprehensive testing for both backend and frontend components
Version Control	Git, GitHub	Source code management and collaboration
CI/CD Pipeline	GitHub Actions, Pre-commit hooks, Black, Flake8, pytest	Automated code quality, testing, and branch protection workflows
Advanced LLM Workflows	LangChain, LangChain-Ollama, LangChain-Chroma, LangChain-Text-Splitters, transformers	RAG implementation, vector storage, text processing, and model tokenization

Layer/Component	Technology / Tool	Purpose
Module-Specific Technologies	KeyBERT, BeautifulSoup4, python-graphviz, py2cfg, staticfg	Keyword extraction, web scraping, control flow visualization, AST analysis
File Upload & Processing	HTML file input, Python file handling	File selection with Python file validation (.py files only)
UI Enhancement	Material-UI Icons, Fontsource Roboto, Emotion styling	Professional UI components and typography

5. Data Flow / Interaction Sequence (Typical Test Generation)

1. User Interaction (Frontend):

- User navigates to the web UI (React app running on `http://localhost:3000`).
- User can upload Python files via file selection (.py files only).
- User inputs a textual prompt (e.g., "Generate unit tests for this function.").
- User selects a specific LLM from the dropdown list showing running status and licensing.
- User optionally configures processing modules via the sidebar, with automatic dependency resolution.
- User submits the request via the send button, which triggers the processing pipeline.

2. Frontend to Backend API:

- The React frontend constructs an HTTP POST request.
- The request is sent to the Backend API's `/prompt` endpoint (e.g., `http://backend:8000/prompt` when running via Docker Compose, or `http://localhost:8000/prompt` if backend is run directly).
- The request body is a JSON object structured according to the `PromptData` Pydantic schema (defined in `backend/schemas.py`), containing the model ID, user message, source code, system message, and generation options.

3. Backend API (`backend/api.py`):

- The FastAPI application receives the `PromptData` object with model selection and module configuration.
- The system automatically discovers and loads the requested modules from the `modules/` directory.
- Module dependencies are resolved automatically, loading prerequisite modules in the correct order.
- The `/prompt` endpoint handler passes the `PromptData` to the `ModuleManager` for pre-processing if any "before" modules are active.
- The (potentially modified) `PromptData` containing the target `model_id` is then passed to the `LLMManager` instance.

4. LLMManager (`backend/llm_manager.py`):

- The `start_model_container(model_id)` method is called (if the model's container isn't already active). This involves:
 - Verifying the `model_id` against `allowed_models.json`.
 - Pulling the base `ollama/ollama` Docker image (if not locally available) with progress tracking.
 - Finding a free host port dynamically.
 - Starting a new Docker container for Ollama with proper network configuration (backend network when running in Docker).
 - Mounting the `backend/ollama-models` volume for model persistence.
 - Waiting for the Ollama API within that new container to become responsive (with timeout protection).
 - Instructing the Ollama instance (via its API) to pull the specific LLM with progress tracking.
- The `send_prompt(prompt_data, ...)` method is called. It:
 - Retrieves the model's context size and trims the prompt if necessary (with special handling for Llama models).
 - Constructs the final prompt string (potentially using `rag_prompt` from `PromptData` or combining user message and source code).
 - Prepares a JSON payload for the Ollama `/api/generate` endpoint, including the model ID, final prompt, system message, and generation options.
 - Makes a streaming HTTP POST request with configurable timeout to the specific Ollama container's API endpoint.
 - Handles timeout scenarios gracefully, returning appropriate error responses.
 - Collects streaming responses and aggregates them into the final output.

5. Ollama Service & LLM:

- The targeted Ollama container receives the generation request.
- Ollama passes the prompt and options to the loaded LLM.
- The LLM processes the input and generates the response (e.g., test code in Markdown format).
- Ollama streams the generated tokens back as a series of JSON objects.

6. LLManager & Backend API (Response Handling):

- `LLManager's send_prompt` method collects the streamed JSON chunks, extracts the `response` content (which forms the Markdown text), and aggregates it.
- It records comprehensive timing metrics (loading time, generation time) and timeout status.
- It constructs a `ResponseData` Pydantic object containing the original model metadata, the LLM's Markdown output, and the timing data.
- This `ResponseData` object is returned to the `api.py` endpoint handler.
- The endpoint handler then passes this `ResponseData` (and the original `PromptData`) to the `ModuleManager` for post-processing if any "after" modules are active.
- The flow in `backend/execution.py` (for CLI usage) saves the final `ResponseData` object as `response.json` in `timestamped outputs/archive/` and `outputs/latest/` directories.
- The `api.py` endpoint returns a comprehensive JSON response to the frontend, including response markdown, timing data, module outputs, complexity metrics, and execution results.

7. Frontend Display:

- The React frontend receives the comprehensive JSON response from the Backend API.
 - It extracts the Markdown content, timing data, module outputs, and complexity metrics.
 - It support to render the Markdown, displaying the LLM-generated test code and any accompanying text.
 - The interface displays additional information in an expandable "Module Output" section when modules are used:
 - Response time (generation time)
 - Code complexity analysis (CCC/MCC for both input and output)
 - Syntax validation status
 - Token count information
 - The chat history maintains context, showing both user inputs (with attached files) and AI responses.
 - Users can shut down model containers to manage resources and monitor real-time model status.
-